

Decode and Analysis algorithm Source document

It contains the how to use the Gstreamer and OpenCV combined with deep learning algorithm package

The analysis algorithm using OpenCV to output the results

```
#ifndef _MSC_VER
#include <sys/time.h>
#define MACRO_COUNT_TIME_START(name) struct timeval
macro_tv_start_##name;\
    struct timeval macro_tv_end_##name;\
    gettimeofday(&macro_tv_start_##name,NULL);
#define MACRO_COUNT_TIME_END(name,___total_count___)
gettimeofday(&macro_tv_end_##name,NULL);\
    printf("%s\t\t\t\t\ttime\t\t\t\t\tcost:\t\t\t\t\t%.2f\t\t\t\t\tms\t\t\t\t\t\n",\t\t\t\t\t#name,\t\t\t\t\t
(
\t\t\t\t\t(double)(macro_tv_end_##name.tv_sec-
macro_tv_start_##name.tv_sec)*1000000+(double)(macro_tv_end_##name.tv_usec-
macro_tv_start_##name.tv_usec) )/1000/___total_count___);
#endif

int analyse(cv::Mat img) {
    time_t nowtime;
    struct tm* p;;
    time(&nowtime);
    p = localtime(&nowtime);
    int frameCount = 0;
    vptdt_result vpt_results;
    char saveimgname[260];
    vpt_results.objcount = 0;
```

```

printf("vpt_version: %s\n", vptdt_get_version());
double totalTime = 0, avgTime = 0;
vptdt_param vptdt_param_;
vptdt_param_.gpuid = 0;
vptdt_param_.det_thre = 0.6;
vptdt_param_.max_batchsize = 10;
vptdt_param_.fusion_interval = 1;
void *handle = NULL;
vptdt_init(&handle, vptdt_param_);
if (img.data == NULL || img.cols == 0 || img.rows == 0) {
    printf("sy_vpt_det_tracker_vdt_process_(error): video  imgdata == null or
w == 0 or h == 0.\n");
    return -1;
}

if (img.data != NULL) {
    if (test_log_flag)
        cout << "----- decode " << frameCount << " -----" <<
endl << endl;
    int res = vptdt_process(handle, img.data, img.cols, img.rows, &vpt_results);
    if (0 == vptdt_process(handle, img.data, img.cols, img.rows, &vpt_results))
    {

        for (int i = 0; i < vpt_results.objcount; ++i) {
            rectangle(img, cv::Point2f(vpt_results.objinfo[i].left,
vpt_results.objinfo[i].top),
cv::Point2f(vpt_results.objinfo[i].right,
vpt_results.objinfo[i].bottom), cv::Scalar(255, 0, 0), 3, 1);
            char str_i[100];
            memset(str_i, 0, 100);
            sprintf(str_i, "%d %.2f %d",
vpt_results.objinfo[i].id, vpt_results.objinfo[i].confidence, vpt_results.objinfo[i].index);
            cv::putText(img, str_i, cv::Point(vpt_results.objinfo[i].left,
vpt_results.objinfo[i].top), cv::FONT_HERSHEY_COMPLEX, 1.3, cv::Scalar(0, 0,
255));

            printf("vptdt
result:%d %d %f %d %d-%d\n", i, vpt_results.objinfo[i].id, vpt_results.objinfo[i].confidence, vpt_results.objinfo[i].index, vpt_results.objinfo[i].left,
vpt_results.objinfo[i].top);
            sprintf(saveimgname, "%s%s%s%s/%02d:%02d:%02d.jpg",
"/home/cjlu/img/" , p->tm_year+1900, p->tm_mon+1, p->tm_day, p->tm_hour, p->tm_min, p->tm_sec);
            cv::imwrite(saveimgname, img);

```

```
        }  
    }  
    else {  
        printf("fail to analyse");  
    }  
    frameCount++;  
}  
}
```

Decode algorithm using Gstreamer and OpenCV

```
using namespace std;
cv::Mat output_img;
#define test_log_flag 1
/* Structure to contain all our information, so we can pass it to callbacks */
typedef struct _CustomData {
    GstElement* pipeline;
    GstElement* source;
    GstElement* depay;
    GstElement* parse;
    GstElement* decoder;
    GstElement* convert;
    GstElement* sink;
    GstElement* cudadownload;
    GstElement* app_queue;
} CustomData;

/* This function will be called by the pad-added signal */
void pad_added_handler(GstElement* src, GstPad* new_pad, CustomData* data) {
    GstPad* sink_pad = gst_element_get_static_pad(data->depay, "sink");
    GstPadLinkReturn ret;
    GstCaps* new_pad_caps = NULL;
    GstStructure* new_pad_struct = NULL;
    const gchar* new_pad_type = NULL;
    g_print("Received new pad '%s' from '%s':\n", GST_PAD_NAME(new_pad),
GST_ELEMENT_NAME(src));
    /* If our converter is already linked, we have nothing to do here */
    if (gst_pad_is_linked(sink_pad)) {
        g_print("We are already linked.\n");
        goto exit;
    }
    /* Check the new pad's type */
    new_pad_caps = gst_pad_get_current_caps(new_pad);
    new_pad_struct = gst_caps_get_structure(new_pad_caps, 0);
    new_pad_type = gst_structure_get_name(new_pad_struct);
    /* Attempt the link */
    ret = gst_pad_link(new_pad, sink_pad);
    if (GST_PAD_LINK_FAILED(ret)) {
        g_print("Type is '%s' but link failed.\n", new_pad_type);
    }
}
```

```

    }
    else {
        g_print("Link succeeded (type '%s').\n", new_pad_type);
    }
    exit:
    /* Unreference the new pad's caps, if we got them */
    if (new_pad_caps != NULL)
        gst_caps_unref(new_pad_caps);
    /* Unreference the sink pad */
    gst_object_unref(sink_pad);
}

static GstFlowReturn ReadVideoFrame_callback(GstElement* sink, gpointer
user_data)
{
    CustomData* data = (CustomData*)user_data;
    GstSample* sample = nullptr;
    GstMapInfo map;
    GstCaps *caps = NULL;
    GstBuffer *buffer = NULL;
    const GstStructure *info = NULL;
    GstFlowReturn ret = GST_FLOW_OK;
    int sample_width = 1920;
    int sample_height = 1080;

    // use pull-sample to get the frame, Map to 'map' variable, use map to copy the
    frame data.
    g_signal_emit_by_name(sink, "pull-sample", &sample);
    gst_buffer_map(buffer, &map, GST_MAP_READ);
    if (ret != GST_FLOW_OK) {
        g_printerr("can't pull GstSample.");
        return ret;
    }
    if (sample) {
        buffer = gst_sample_get_buffer(sample);
        if (buffer == NULL) {
            g_printerr("get buffer is null");
            goto exit;
        }
        gst_buffer_map(buffer, &map, GST_MAP_READ);
        caps = gst_sample_get_caps(sample);
        if (caps == NULL) {
            g_printerr("get caps is null");
            goto exit;
        }
    }
}

```

```

    }
    info = gst_caps_get_structure(caps, 0);
    if (info == NULL) {
        g_printerr("get info is null");
        goto exit;
    }
    if (map.data == NULL) {
        g_printerr("appsink buffer data empty\n");
        goto exit;
    }
    gst_structure_get_int(info, "width", &sample_width);
    gst_structure_get_int(info, "height", &sample_height);
    cv::Mat rgbImg(sample_height, sample_width, CV_8UC3,
        (unsigned char *) map.data, cv::Mat::AUTO_STEP);
    rgbImg.copyTo(output_img);

}

exit:
if (buffer) {
    gst_buffer_unmap(buffer, &map);
    printf("no buffer");
}
if (sample) {
    gst_sample_unref(sample);
    printf("no sample");
}
return GST_FLOW_OK;
}

```

```

int decoder_init(){
    GstCaps *video_caps;
    CustomData data;
    GstBus* bus;
    GstMessage* msg;
    GstStateChangeReturn ret;
    gboolean terminate = FALSE;
    /* Initialize GStreamer */
    /* Create the elements */
    data.source = gst_element_factory_make("rtspsrc", "source");
    g_object_set(G_OBJECT(data.source), "latency", 2000, NULL);
    data.depay = gst_element_factory_make("rtph264depay", "depay");
    data.parse = gst_element_factory_make("h264parse", "parse");
    data.decoder = gst_element_factory_make("nvh264dec", "decoder");
}

```

```

data.convert = gst_element_factory_make("cudaconvert", "convert");
data.cudadownload = gst_element_factory_make("cudadownload",
"cudadownload");
data.app_queue = gst_element_factory_make ("queue", "app_queue");
data.sink = gst_element_factory_make("appsink", "sink");
if (!data.decoder || !data.convert || !data.cudadownload)//If no GPU to use, using the
softdecode.
{
    data.decoder = gst_element_factory_make("avdec_h264", "decoder");
    data.convert = gst_element_factory_make("videoconvert", "cudaconvert");
    data.cudadownload = gst_element_factory_make("queue", "cudadownload");
    g_print("NO NVIDIA GPU to use! Use SoftDecode\n");
}
else
{
    g_print("Use HardDecode!\n");
}
video_caps = gst_caps_new_simple("video/x-raw",
    "width", G_TYPE_INT, 1920,
    "height", G_TYPE_INT, 1080,
    "framerate", GST_TYPE_FRACTION, 0, 1,
    "format", G_TYPE_STRING, "BGR", NULL);

/* Set the URI to play */
g_object_set(data.source,
"location","rtsp://admin:admin123@192.168.1.192/h264/ch1/main/av_stream",
NULL);
/* Connect to the pad-added signal */
g_signal_connect(data.source, "pad-added",
G_CALLBACK(pad_added_handler), &data);
g_object_set (data.sink, "caps", video_caps, NULL);
g_object_set(data.sink, "emit-signals", TRUE, NULL);
g_signal_connect(data.sink, "new-sample",
G_CALLBACK(ReadVideoFrame_callback), &data);
/* Create the empty pipeline */
data.pipeline = gst_pipeline_new("test-pipeline");
if (!data.pipeline || !data.decoder || !data.parse || !data.depay || !data.source
|| !data.convert || !data.cudadownload || !data.app_queue || !data.sink) {
    g_printerr("Not all elements could be created.\n");
    return -1;
}

/* Build the pipeline. If we are NOT linking the source at this point. We will do
it later. */

```

```

    gst_bin_add_many(GST_BIN(data.pipeline), data.source, data.depay, data.parse,
data.decoder, data.convert,data.cudadownload,data.app_queue, data.sink, NULL);

    if (!gst_element_link_many(data.depay, data.parse, data.decoder, data.convert,
data.cudadownload,data.app_queue, data.sink, NULL)) {
        g_printerr("Elements could not be linked.\n");
        gst_object_unref(data.pipeline);
        return -1;
    }
    /* Start playing */
    ret = gst_element_set_state(data.pipeline, GST_STATE_PLAYING);

    if (ret == GST_STATE_CHANGE_FAILURE) {
        g_printerr("Unable to set the pipeline to the playing state.\n");
        gst_object_unref(data.pipeline);
        return -1;
    }
    /* Listen to the bus */
    bus = gst_element_get_bus(data.pipeline);

    do {

        msg =
            gst_bus_timed_pop_filtered(bus, GST_CLOCK_TIME_NONE,

(GstMessageType)(GST_MESSAGE_STATE_CHANGED
GST_MESSAGE_ERROR | GST_MESSAGE_EOS));

        /* Parse message */
        if (msg != NULL) {
            GError* err;
            gchar* debug_info;
            switch (GST_MESSAGE_TYPE(msg)) {
                case GST_MESSAGE_ERROR:
                    gst_message_parse_error(msg, &err, &debug_info);
                    g_printerr("Error received from element %s: %s\n",
GST_OBJECT_NAME(msg->src), err->message);
                    g_printerr("Debugging information: %s\n", debug_info ?
debug_info : "none");
                    g_clear_error(&err);
                    g_free(debug_info);
                    terminate = TRUE;
                    break;
                case GST_MESSAGE_EOS:

```



```

        g_print("End-Of-Stream reached.\n");
        terminate = TRUE;
        break;
    case GST_MESSAGE_STATE_CHANGED:
        if (GST_MESSAGE_SRC(msg) ==
GST_OBJECT(data.pipeline)) {
            GstState old_state, new_state, pending_state;
            gst_message_parse_state_changed(msg, &old_state,
&new_state, &pending_state);
            g_print("Pipeline state changed from %s to %s:\n",
gst_element_state_get_name(old_state),
gst_element_state_get_name(new_state));
        }
        break;
    default:
        g_printerr("Unexpected message received.\n");
        break;
    }
    gst_message_unref(msg);
}
} while (!terminate);
gst_object_unref(bus);
gst_element_set_state(data.pipeline, GST_STATE_NULL);
gst_object_unref(data.pipeline);
/* Free resources */
return 0;
}

```

The import deep learning in the analysis algorithm

```

#ifndef _MSC_VER
#ifdef VPTDETTRACK_EXPORTS
#define VPTDETTRACK_API __declspec(dllexport)
#else
#define VPTDETTRACK_API __declspec(dllimport)
#endif
#else
#define VPTDETTRACK_API __attribute__((visibility("default")))

```

```

#include <stdlib.h>
#endif

#include <vector>
#include <iostream>
#include "vpt_head.h"

#ifdef __cplusplus
extern "C"
{
#endif

#ifndef VPTDETTRACK_PARAM
#define VPTDETTRACK_PARAM
typedef struct vptdt_param
{
    int gpuid;          //Get GPU
    float det_thre;     // Set threshold, Default is 0.6
    int max_batchsize; // Used for opening up GPU memory

    int fusion_interval; //1: Detect every frame 2: skip one frame detection and
so on
    vptdt_param() : gpuid(0), det_thre(0.6), max_batchsize(10), fusion_interval(1) {};
} vptdt_param;
#endif

/*****
*****
* FUNCTION: vptdt_init
* PURPOSE: initialization
* PARAM:
[in] handle
[in] param      - initialization
* RETURN: success(0) or error code(<0)
*****/
VPTDETTRACK_API int vptdt_init(void **handle, vptdt_param param);

/*****
*****
* FUNCTION: vptdt_release
* PURPOSE: release data
* PARAM:

```

```

[in] handle
* RETURN: NULL
* NOTES:
*****
****/
VPTDETRACK_API void vptdt_release(void **handle);

/*****
*****
* FUNCTION: vptdt_process
* PURPOSE: detect people and car in the video
* PARAM:
    [in] handle      - Handle to load the process
    [in] rgb          - Image format
    [in] width
    [in] height
    [in] result       - detect result
* RETURN: count(>= 0) or false(<0)
* NOTES:
*****
****/
VPTDETRACK_API int vptdt_process(void * handle, unsigned char * rgb, int width,
int height, vptdt_result * result);
VPTDETRACK_API int vptdt_process_batch(void * handle, xjs_batch_img
*img_data_array, int batchsize, vptdt_result * result);

/*****
*****
* FUNCTION: vptdt_get_version
* PURPOSE: Get Version
* PARAM: NULL
* RETURN: Version
* NOTES:
*****
****/
VPTDETRACK_API const char * vptdt_get_version();
#ifdef __cplusplus
};
#endif

#ifndef __XJS_IMG__
#define __XJS_IMG__

```

```

typedef struct xjs_batch_img
{
    int w_;
    int h_;
    int c_;
    unsigned char * data_;
    //std::string video_key_;
    const char* video_key_;
    long frameCount_;
    void set_data(int m_w, int m_h, int m_c, unsigned char * m_data, const char*
m_video_key, long fcnt_)
    {
        w_ = m_w;
        h_ = m_h;
        c_ = m_c;
        data_ = m_data;
        video_key_ = m_video_key;
        frameCount_ = fcnt_;
    }
} xjs_batch_img;
#endif

```

```

//----- Structure definition of analysis results for people, vehicles, objects, and
urban management -----//
#define MAX_OBJ_COUNT 100

```

```

//Rect
#ifndef __SY_RECT__
#define __SY_RECT__
typedef struct sy_rect
{
    int left_;
    int top_;
    int width_;
    int height_;
    sy_rect(int m_left, int m_top, int m_width, int m_height) :left_(m_left),
top_(m_top), width_(m_width), height_(m_height) {};
    sy_rect() {};
} sy_rect;
#endif

```

```

#ifndef VD_RESULT_
#define VD_RESULT_
typedef struct vd_result
{
    sy_rect rect;
    float score;
}vd_result;
#endif

```

```

#ifndef VPTDT_INFO
#define VPTDT_INFO
typedef struct vptdt_info //struct the result
{
    int left;
    int top;
    int right;
    int bottom;
    int center_x;
    int center_y;
    int id;           // same ID will be recognized as the same one
    double confidence; // confidence of detected object

    int index;        //class: 0-person 1-bike 2-motor 3- tricycle 4-car 5-
bigbus 6-lorry 7-tractor 8-midibus

```

```

    vd_result w_res;
    vd_result b_res;
}vptdt_info;
#endif

```

```

#ifndef VPTDT_RESULT
#define VPTDT_RESULT
typedef struct vptdt_result
{
    vptdt_info objinfo[MAX_OBJ_COUNT];
    int objcount;
}vptdt_result;
#endif

```